Football League Database

Technical Documentation

Rockborne mini-project — Task 2: implementation and testing

Hassan Hashmi

August 2026

1. Project goals

This project delivers a relational database for a six-a-side football league. The league runs through the autumn, with four teams drawn from eight consultant groups playing each other home and away across a six-week season, and the side with the most points at the end is the winner.

The database records the teams, players, groups, venues, fixtures, results, goals and cards from each match. It also supports friendly matches, which sit outside the league but need to be stored in the same structure.

Task 1 produced the design as an entity relationship model in third normal form. This document covers Task 2: building that design as a working PostgreSQL database on a local server, loading it with the season's data, and testing that it answers the questions the scenario asks - for example the league table, the leading scorers and the disciplinary record.

Two aims carried over from the Task 1 feedback and shape the work here: to enforce, in the database itself, two rules that the original design left to careful data entry; and to extend the data dictionary so that every field states its type, whether it may be empty, and the values it accepts.

2. Implementation steps

The database was built on a local PostgreSQL server through pgAdmin, in the following order.

1. Created a dedicated database, football_league, so the project sits in its own namespace on the local server rather than the shared AWS instance.
2. Ran the schema script to build the structure: four domains for the repeated value rules, the three reference tables (teams, groups and venues), the players table, the matches table, and the goals and cards tables, followed by the two integrity triggers.
3. Ran the data script to populate every table with the season's data. The matches load first with empty scores, then the goals and cards, and a final update sets the scores once the goal rows are in place. Loading in this order lets the score trigger check the data instead of blocking the load.
4. Generated the entity relationship diagram from the finished schema using the ERD tool in pgAdmin.
5. Ran the seven test queries to confirm the database returns the expected results.

3. Updates to the design

The structure follows the Task 1 design. Three changes were made in response to the feedback on that submission: two new integrity controls and the extended data dictionary given in the next section. No tables or keys were restructured and the design remains in third normal form.

Event players must belong to the match

In the original design nothing prevented a goal or a card being recorded against a player who did not appear in that match. A trigger now checks, whenever a goal or card is added or changed, that the player's team is one of the two teams in the match and rejects the entry if it is not. This rule spans three tables - the player, the match and the two teams - which is why it is handled by a trigger rather than a simple constraint.

Stored scores kept in step with the goals

The home and away scores held on each match repeat a total that the goal rows already contain. A second trigger keeps the two in agreement: whenever a goal is added, changed or removed, or a score is edited, it recounts the goals for each team and rejects any score that does not match. Matches that have not been played, which carry empty scores and have no goals, are left alone.

Domains and constraints

The repeated value rules are defined once as PostgreSQL domains and reused across the tables: the match type (league or friendly), the card colour (yellow or red), the minute (a value between 1 and 120) and the score (zero or more). Two further check constraints complete the design - a team cannot be both the home and away side in the same match and a match's two scores must either both be present or both be empty. Together these mean the database enforces by itself the rules that the Task 1 data met by hand.

The full definitions of the domains, constraints and triggers are given in Appendix A.

4. Data dictionary

The dictionary lists every field, grouped by table, with its meaning, its type, whether it may be left empty, and the values it accepts. Where a field appears in more than one table, such as a team or player reference, it is listed under each table it belongs to.

Teams

- team_id: unique identifier for the team and primary key. Integer, generated automatically, never empty.
- team_name: name of the team, such as Data Masters or BI Gods. Text of up to 100 characters, never empty, and must be unique.

Groups

- group_id: unique identifier for the group and primary key. Integer, generated automatically, never empty.
- group_name: name of the consultant group a player belongs to, such as Cohort 4 or HR. Text of up to 100 characters, never empty, and must be unique.

Venues

- venue_id: unique identifier for the venue and primary key. Integer, generated automatically, never empty.
- venue_name: name of the pitch a match is played on, such as Wimbledon 1. Text of up to 100 characters, never empty, and must be unique.

Players

- player_id: unique identifier for the player and primary key. Integer, generated automatically, never empty. It replaces the consultant identifier in the source data, which was shared by two different people and so could not serve as a key.
- first_name: the player's first name. Text of up to 50 characters, never empty.
- last_name: the player's last name. Text of up to 50 characters, never empty.
- team_id: the team the player belongs to; foreign key to teams. Integer, never empty. Each player belongs to exactly one team.
- group_id: the group the player belongs to; foreign key to groups. Integer, never empty.

Matches

- match_id: unique identifier for the match and primary key. Integer, generated automatically, never empty.
- match_datetime: the date and time the match is scheduled for or was played. Timestamp, never empty.
- home_team_id: the team playing at home; foreign key to teams. Integer, never empty.

- away_team_id: the team playing away; foreign key to teams. Integer, never empty, and must differ from the home team.
- venue_id: the venue the match is played at; foreign key to venues. Integer, never empty.
- match_type: whether the fixture is a league match or a friendly. Text, never empty, and limited to the values league or friendly.
- home_score: goals scored by the home team, left empty until the match has been played. Small integer, zero or more, and empty only when the away score is also empty.
- away_score: goals scored by the away team, left empty until the match has been played. Small integer, zero or more, and empty only when the home score is also empty.

Goals

- goal_id: unique identifier for the goal and primary key. Integer, generated automatically, never empty.
- match_id: the match the goal was scored in; foreign key to matches. Integer, never empty.
- player_id: the player who scored the goal; foreign key to players. Integer, never empty, and must be a player from one of the two teams in the match.
- minute: the minute of the match the goal was scored in. Small integer between 1 and 120, never empty.

Cards

- card_id: unique identifier for the card and primary key. Integer, generated automatically, never empty.
- match_id: the match the card was issued in; foreign key to matches. Integer, never empty.
- player_id: the player who received the card; foreign key to players. Integer, never empty, and must be a player from one of the two teams in the match.
- card_type: whether the card is yellow or red. Text, never empty, and limited to the values yellow or red.
- minute: the minute of the match the card was issued in. Small integer between 1 and 120, never empty.

5. Entity relationship diagram

The ERD was produced by pgAdmin directly from the built schema. It shows the seven tables with their primary keys, and the foreign key relationships in crow's-foot notation - including the two separate links from matches to teams for the home and away roles. The domains appear as the column types, for example match_type and score_value. The check constraints and triggers are enforced by the database and are not shown on the diagram.



6. Database scripts

The database can be rebuilt from two scripts, supplied with this document and listed in full in the appendices:

- schema_ddl.sql creates the domains, tables, keys, constraints and triggers (Appendix A).
- data_dml.sql populates every table with the season's data (Appendix B).

Run the schema script first, then the data script, against a local PostgreSQL database. The seven test queries are held separately in test_queries.sql and are shown in the testing section below. A README file is included with the submission giving the full run order and the prerequisites.

6. Testing

Each of the seven test cases from the brief is set out below. For every test there is a short statement of its purpose, the query used (with the expected outcome noted in a comment at the top), a screenshot of the result of the query from pgAdmin and a brief statement of whether the test passed.

Test 1 — Players in a given group (department)

Purpose. Lists every player in a chosen group, here Cohort 4, together with the team they play for.

```
-- Test 1: list all the players that play for a particular department
-- Expected: 2 rows - Aedan Petty (Data Masters), Kyleigh Phelps (BI Gods)
SELECT p.player_id, p.first_name, p.last_name, t.team_name, g.group_name
FROM players AS p
JOIN groups AS g ON g.group_id = p.group_id  -- obtain group names via a group_id join
JOIN teams AS t ON t.team id  = p.team id    -- obtain team names via a team id join
WHERE g.group name = 'Cohort 4'
ORDER BY p.player id;
```

	player_id integer	first_name character varying (50)	last_name character varying (50)	team_name character varying (100)	group_name character varying (100)
1	1	Aedan	Petty	Data Masters	Cohort 4
2	13	Kyleigh	Phelps	BI Gods	Cohort 4

Success. The two Cohort 4 players are returned with their correct teams, so the test passes.

Test 2 — Fixtures on a given date

Purpose. Lists every fixture on a chosen date, here 29 October 2022, with both team names and the venue.

```
-- Test 2: list all the fixtures for a particular date (e.g. 29th Oct 2022)
-- Expected: 2 rows - matches 7 and 8 on 2022-10-29
SELECT m.match_id,
       m.match datetime,
       home.team_name AS home_team,   -- teams joined once for the home side
       away.team_name AS away_team,   -- and again for the away side
       v.venue_name,
       m.home_score,
       m.away score
FROM matches AS m
JOIN teams AS home ON home.team id = m.home team id
JOIN teams AS away ON away.team_id = m.away_team_id
JOIN venues AS v ON v.venue id = m.venue id
WHERE m.match datetime::date = DATE '2022-10-29'   -- compare on the date only, ignoring the time
ORDER BY m.match id;
```

	match_id integer	match_datetime timestamp without time zone	home_team character varying (100)	away_team character varying (100)	venue_name character varying (100)	home_score smallint	away_score smallint
1	7	2022-10-29 16:00:00	BI Gods	Data Masters	Wimbledon 1	2	2
2	8	2022-10-29 16:00:00	Data Cleaners	Vis Wizards	Wimbledon 3	1	2

Success. Both fixtures for the date are returned with the right teams and venues, so the test passes.

Test 3 — Players with more than two goals

Purpose. Lists the players who scored more than two goals across the season, with their goal count.

```
-- Test 3: list all players who scored more than 2 goals
-- Expected: 4 rows - Ali 5, Dodson 5, Cochran 4, Stevenson 4
SELECT p.player_id,
       p.first name,
       p.last_name,
       COUNT(*) AS goals_scored        -- one row per goal, so COUNT(*) is used here
FROM goals AS g
JOIN players AS p ON p.player id = g.player id
GROUP BY p.player id, p.first name, p.last name
HAVING COUNT(*) > 2                     -- HAVING filters the grouped totals
ORDER BY goals_scored DESC, p.last_name;
```

	player_id [PK] integer	first_name character varying (50)	last_name character varying (50)	goals_scored bigint
1	8	Alvin	Ali	5
2	19	Jamya	Dodson	5
3	35	Dominik	Cochran	4
4	27	Hayley	Stevenson	4

Success. The four players above two goals are returned with the correct counts, so the test passes.

Test 4 — Cards per team

Purpose. Counts the yellow and red cards for each team, with an overall total.

```
--  Test 4: List the number of cards (yellow and red) per team
-- Expected: Data Masters 5/0, BI Gods 2/0, Vis Wizards 2/0, Data Cleaners 2/2 (yellow/red)
SELECT t.team name,
       COUNT(*) FILTER (WHERE c.card type = 'yellow') AS yellow cards,
       COUNT(*) FILTER (WHERE c.card_type = 'red') AS red_cards,   -- count each colour separately
       COUNT(*) AS total_cards
FROM cards AS c
JOIN players AS p ON p.player id = c.player id
JOIN teams AS t ON t.team id = p.team id
GROUP BY t.team_name
ORDER BY t.team name;
```

	team_name character varying (100)	yellow_cards bigint	red_cards bigint	total_cards bigint
1	BI Gods	2	0	2
2	Data Cleaners	2	2	4
3	Data Masters	5	0	5
4	Vis Wizards	2	0	2

Success. Every team's card counts match the results and total thirteen, so the test passes.

Test 5 — Fixtures still to be played

Purpose. Lists the fixtures that have not yet been played, which are the two friendlies.

```
-- Test 5: List the games that are going to be played (friendly matches)
-- Expected: 2 rows - matches 13 and 14 (unplayed friendlies)
SELECT m.match_id,
       m.match_datetime,
       home.team_name AS home_team,
       away.team_name AS away_team,
       v.venue_name
FROM matches AS m
JOIN teams AS home ON home.team_id = m.home_team_id
JOIN teams AS away ON away.team_id = m.away_team_id
JOIN venues AS v ON v.venue_id = m.venue_id
WHERE m.match_type = 'friendly'    -- friendlies only
  AND m.home_score IS NULL         -- use AND condition to obtain rows that satisfy both conditions
ORDER BY m.match_datetime;
```

	match_id integer	match_datetime timestamp without time zone	home_team character varying (100)	away_team character varying (100)	venue_name character varying (100)
1	13	2023-11-12 16:00:00	Data Cleaners	Data Masters	Wimbledon 1
2	14	2023-11-12 16:00:00	Vis Wizards	BI Gods	Wimbledon 3

Success. Both unplayed friendlies are returned and no completed fixture appears, so the test passes.

Test 6 — League table

Purpose. Shows each team with its total league points, awarding three for a win, one for a draw and none for a loss.

```
--  Test 6: Obtain a table displaying the team's name and the points earned during the tournament
-- Expected: Vis Wizards 11, Data Masters 8, BI Gods 8, Data Cleaners 5
WITH results AS (
    -- points earned by each team in its HOME fixtures
    SELECT home_team_id AS team_id,
           CASE WHEN home_score > away_score THEN 3   -- home win
                WHEN home_score = away_score THEN 1   -- draw
                ELSE 0 END AS points                  -- home loss
    FROM matches
    WHERE match_type = 'league' AND home_score IS NOT NULL  -- exclude friendlies
    UNION ALL   -- stack the away results (below) beneath the home results, duplicates included
    -- points earned by each team in its AWAY fixtures
    SELECT away_team_id AS team_id,
           CASE WHEN away_score > home_score THEN 3   -- away win
                WHEN away_score = home_score THEN 1   -- draw
                ELSE 0 END AS points
    FROM matches
    WHERE match_type = 'league' AND home_score IS NOT NULL
)
SELECT t.team_name,
       SUM(r.points) AS points
FROM teams AS t
JOIN results AS r ON r.team_id = t.team_id
GROUP BY t.team_name
ORDER BY points DESC, t.team_name;
```

	team_name character varying (100)	points bigint
1	Vis Wizards	11
2	BI Gods	8
3	Data Masters	8
4	Data Cleaners	5

Success. The points total thirty-two and place Vis Wizards top, matching a manual check, so the test passes.

Test 7 — Goals scored and conceded per half

Purpose. For each team, shows goals scored and goals conceded, split between the first and second half of the match.

```
-- Test 7: List the goals scored and conceded, per team, per half
-- Expected: 4 rows, scored columns total 32 and conceded columns total 32
WITH goal_detail AS
(
    SELECT p.team_id AS scoring_team,                          -- the team that scored
           CASE WHEN p.team_id = m.home_team_id                -- if the scorer is the home team,
                THEN m.away_team_id                            -- then the away team conceded
                ELSE m.home_team_id END AS conceding_team,  -- otherwise home team conceded the goal
           CASE WHEN g.minute <= 45
                THEN 'First half'                             -- categorise each goal by minute
                ELSE 'Second half' END AS half
    FROM goals AS g
    JOIN players AS p ON p.player_id = g.player_id    -- obtain the team belonging to the scorer
    JOIN matches AS m ON m.match_id  = g.match_id     -- obtain the match containing the goal
    WHERE m.match_type = 'league'
)
SELECT t.team_name,
       COUNT(*) FILTER
                      (WHERE gd.scoring_team = t.team_id AND gd.half = 'First half') AS
scored_first_half,
       COUNT(*) FILTER
                      (WHERE gd.scoring_team = t.team_id AND gd.half = 'Second half') AS
scored_second_half,
       COUNT(*) FILTER
                      (WHERE gd.conceding_team = t.team_id AND gd.half = 'First half') AS
conceded_first_half,
       COUNT(*) FILTER
                      (WHERE gd.conceding_team = t.team_id AND gd.half = 'Second half') AS
conceded_second_half
FROM teams AS t
LEFT JOIN goal_detail AS gd
       ON t.team_id IN (gd.scoring_team, gd.conceding_team)   -- any goal in a match the team played
GROUP BY t.team_name, t.team_id
ORDER BY t.team_name;
```

	team_name character varying (100)	scored_first_half bigint	scored_second_half bigint	conceded_first_half bigint	conceded_second_half bigint
1	BI Gods	5	3	4	5
2	Data Cleaners	3	2	4	2
3	Data Masters	3	5	4	4
4	Vis Wizards	5	6	4	5

Success. The scored and conceded columns each total thirty-two, so the test passes.

Appendix A — Schema definition (DDL)

This is the full content of schema_ddl.sql.

```
-- CREATE THE DATABASE
CREATE DATABASE football_league;

-- CREATE THE THREE MAJOR TABLES
/*
        Brief explanation pertaining to the tables below -
        - 'INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY':
                - 'PRIMARY KEY': uniquely identifies each row; values cannot be NULL or duplicated.
                - 'GENERATED.. AS IDENTITY': auto-generates an integer for each row
                - 'BY DEFAULT': database generates value unless user provides one.
        - 'CONSTRAINT ... CHECK (btrim(team_name) <> '')':
                - 'CHECK' tells the database that 'the following condition must be true for every
row'
                - btrim() just removes whitespace from beginning and end of the string
                - <> '' checks its not equal to an empty string
*/
CREATE TABLE teams (
    team_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
        -- every row must have a value, and no two values in this column can be the same
    team_name VARCHAR(100) NOT NULL UNIQUE,
    CONSTRAINT teams_team_name_not_blank
                CHECK (btrim(team_name) <> '')  -- ensure the team name is a string containing
significant chars
);
CREATE TABLE groups (
    group_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    group_name VARCHAR(100) NOT NULL UNIQUE,
    CONSTRAINT groups_group_name_not_blank CHECK (btrim(group_name) <> '')
);
CREATE TABLE venues (
    venue_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    venue_name VARCHAR(100) NOT NULL UNIQUE,
    CONSTRAINT venues_venue_name_not_blank CHECK (btrim(venue_name) <> '')
);


-- DEFINE DOMAINS
/*
        Domain: defines a set of allowed values for columns that use it,
        including the underlying data type and any constraints. A domain
        is essentially a custom data type with constraints. We define the
        domain below and can later use it as a column's data type. Pros:
        domains can be re-used every single time without having to impose
        constraints over and over.
*/
CREATE DOMAIN match_type AS VARCHAR(10)
        CHECK (VALUE IN ('league', 'friendly'));
CREATE DOMAIN card_type AS VARCHAR(10)
        CHECK (VALUE IN ('yellow', 'red'));
CREATE DOMAIN match_minute AS SMALLINT
        CHECK (VALUE BETWEEN 1 AND 120);
CREATE DOMAIN score_value AS SMALLINT
        CHECK (VALUE >= 0);


-- CREATE THE SMALLER TABLES REFERENCING THE BIGGER TABLES
-- TABLE 1
CREATE TABLE players (
    player_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    team_id INTEGER NOT NULL REFERENCES teams(team_id),
    group_id INTEGER NOT NULL REFERENCES groups(group_id),
    CONSTRAINT players_first_name_not_blank CHECK (btrim(first_name) <> ''),
    CONSTRAINT players_last_name_not_blank CHECK (btrim(last_name) <> '')
);
/*
    player_id provides a unique identifier for each player.
    team_id is a foreign key, ensuring that every player's team
    exists in the teams table. Multiple players can belong to
    the same team.
*/

-- TABLE 2
/*
```

```
 We create two separate columns for home and away teams, and specifically
 create them as foreign keys that reference the original team id column. The
 values here do not have to be unique, as long as they exist in the primary key
 column that they are referencing, which in this case means that a given team
 can appear in more than 1 match which makes sense.
*/
CREATE TABLE matches (
    match_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    match_datetime TIMESTAMP NOT NULL,
    home_team_id INTEGER NOT NULL REFERENCES teams(team_id),
    away_team_id INTEGER NOT NULL REFERENCES teams(team_id),
    venue_id INTEGER NOT NULL REFERENCES venues(venue_id),
    match_type match_type NOT NULL,
    home_score score_value,
    away_score score_value,
    CONSTRAINT matches_teams_differ
        CHECK (home_team_id <> away_team_id),  -- a team cannot play against itself
        /*
        Both scores must either be recorded or both be NULL.
        This prevents half-recorded results where only one team
        has a score.
    */
    CONSTRAINT matches_scores_both_or_neither
        CHECK ((home_score IS NULL) = (away_score IS NULL))
);

-- TABLE 3
/*
    GOALS:
    Each row represents one goal scored in a match.

    match_id is a foreign key referencing matches(match_id).
    ON DELETE CASCADE means that if a match is deleted, all goals
    belonging to that match are automatically deleted as well.

    minute uses the match_minute domain defined earlier. Therefore,
    the column is called 'minute', while 'match_minute' is the
    custom data type that defines its allowed values.
*/
CREATE TABLE goals (
    goal_id   INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    match_id  INTEGER NOT NULL REFERENCES matches(match_id) ON DELETE CASCADE,
    player_id INTEGER NOT NULL REFERENCES players(player_id),
    minute match_minute NOT NULL
);

-- TABLE 4
CREATE TABLE cards (
    card_id   INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    match_id  INTEGER NOT NULL REFERENCES matches(match_id) ON DELETE CASCADE,
    player_id INTEGER NOT NULL REFERENCES players(player_id),
    card_type card_type NOT NULL,  -- using our card type created domain
    minute    match_minute NOT NULL
);


-- WORKING ON FEEDBACK FROM PART 1 SUBMISSION

/*
    INTEGRITY CONTROL 1:
    The schema does not otherwise enforce that an event player belongs
    to one of the teams actually playing in the match.

    The foreign keys in goals and cards only verify that:
        - match_id exists in matches(match_id)
        - player_id exists in players(player_id)

    However, this does NOT verify the relationship between the player
    and the specific match.

    For example, player 70 may exist and match 1 may exist, but if
    player 70 belongs to team 3 while match 1 is team 1 vs team 2,
    the foreign keys alone would still allow the goal.

    The trigger function below checks that the player's team is either
    the home or away team for the relevant match.
*/
CREATE OR REPLACE FUNCTION enforce_event_player_in_match()
/*
    The function is a trigger function, meaning it is called
```

```
    automatically by a trigger when the specified database event occurs.
*/
RETURNS TRIGGER AS $$
/*
    Store the player's team and the two teams involved in the match
    so that they can be compared.
*/
DECLARE
    v_player_team INTEGER;
    v_home_team   INTEGER;
    v_away_team   INTEGER;
BEGIN
       -- Find the team that the player belongs to.
       /*
           NEW refers to the row being inserted or the new version of a row
           being updated. OLD refers to the existing version of a row,
           particularly relevant when a row is deleted (seen later).
       */
    SELECT team_id INTO v_player_team
      FROM players  WHERE player_id = NEW.player_id;
       -- Find the home and away teams for the relevant match.
    SELECT home_team_id, away_team_id INTO v_home_team, v_away_team
      FROM matches  WHERE match_id  = NEW.match_id;
       /*
        Reject the row if the player's team is neither the home team
        nor the away team.
        IS DISTINCT FROM is used so that NULL values are also handled
        safely in the comparison.
    */
    IF v_player_team IS DISTINCT FROM v_home_team
       AND v_player_team IS DISTINCT FROM v_away_team THEN
        RAISE EXCEPTION -- reject the operation and display the following error message
          'Player % (team %) did not play in match % (teams %/%)',
          NEW.player_id, v_player_team, NEW.match_id, v_home_team, v_away_team;
    END IF;
       -- If the check passes, allow the row to be inserted/updated.
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER goals_check_player
    BEFORE INSERT OR UPDATE ON goals
    FOR EACH ROW EXECUTE FUNCTION enforce_event_player_in_match();
CREATE TRIGGER cards_check_player
    BEFORE INSERT OR UPDATE ON cards
    FOR EACH ROW EXECUTE FUNCTION enforce_event_player_in_match();

/*
    INTEGRITY CONTROL 2:
    '...duplicate totals derivable from the goal rows without a
    synchronisation rule.'
    The problem being solved is that the score stored in the matches
    table can potentially disagree with the goals recorded in the
    goals table.
    For example, a match could have home_score = 2 and away_score = 1,
    while only two goal rows have actually been recorded.
    The function below checks that the stored match score always agrees
    with the number of goals recorded for each team.
*/
CREATE OR REPLACE FUNCTION enforce_score_sync()
/*
    This is a trigger function, so it is automatically called by
    the triggers defined below whenever the relevant table changes.
*/
RETURNS TRIGGER AS $$
/*
    Variables used to store:
        - the relevant match ID
        - the home and away teams
        - the stored home and away scores
        - the number of recorded goals for each team
*/
DECLARE
    v_match_id   INTEGER;
    v_home_team  INTEGER;
    v_away_team  INTEGER;
    v_home_score SMALLINT;
    v_away_score SMALLINT;
    v_home_goals INTEGER;
    v_away_goals INTEGER;
BEGIN
```

```
       /*
        Determine which match is being checked.
        If the trigger fired on the matches table, NEW.match_id
        identifies the match being inserted/updated.
        Otherwise, the trigger fired on goals, so use the match id
        from the new row where available, or the old row when a
        goal is being deleted.
            TG_TABLE_NAME contains the name of the table that caused the trigger
            to execute. This function is used by triggers on both matches and goals.
       */
IF TG_TABLE_NAME = 'matches' THEN
    IF TG_TABLE_NAME = 'matches' THEN
        v_match_id := NEW.match_id;
    ELSE
        v_match_id := COALESCE(NEW.match_id, OLD.match_id);
    END IF;
    SELECT home_team_id, away_team_id, home_score, away_score
      INTO v_home_team, v_away_team, v_home_score, v_away_score
      FROM matches WHERE match_id = v_match_id;
    IF NOT FOUND THEN RETURN COALESCE(NEW, OLD); END IF;       -- match cascaded away
    IF v_home_score IS NULL AND v_away_score IS NULL THEN
        RETURN COALESCE(NEW, OLD);                             -- unplayed: skip
    END IF;
    SELECT COUNT(*) FILTER (WHERE p.team_id = v_home_team),
           COUNT(*) FILTER (WHERE p.team_id = v_away_team)
      INTO v_home_goals, v_away_goals
         -- Join goals to players so we can determine which team scored each goal.
      FROM goals g JOIN players p ON p.player_id = g.player_id
     WHERE g.match_id = v_match_id;
    IF v_home_score <> v_home_goals OR v_away_score <> v_away_goals THEN
        RAISE EXCEPTION
          'Score out of sync for match %: stored %-%, goal rows give %-%',
          v_match_id, v_home_score, v_away_score, v_home_goals, v_away_goals;
    END IF;
    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER goals_sync_score
    AFTER INSERT OR UPDATE OR DELETE ON goals
    FOR EACH ROW EXECUTE FUNCTION enforce_score_sync();
CREATE TRIGGER matches_sync_score
    AFTER INSERT OR UPDATE ON matches
    FOR EACH ROW EXECUTE FUNCTION enforce_score_sync();
```

Appendix B — Data population (DML)

This is the full content of data_dml.sql.

```
-- LOADING THE DATA MANUALLY EXACTLY AS IN EXCEL
INSERT INTO teams (team_id, team_name) VALUES
    (1, 'Data Masters'),
    (2, 'BI Gods'),
    (3, 'Vis Wizards'),
    (4, 'Data Cleaners');
INSERT INTO groups (group_id, group_name) VALUES
    (1, 'Cohort 4'),
    (2, 'Cohort 5'),
    (3, 'Cohort 6'),
    (4, 'Cohort 7'),
    (5, 'Bench'),
    (6, 'Training Team'),
    (7, 'HR'),
    (8, 'Consultants');
INSERT INTO venues (venue_id, venue_name) VALUES
    (1, 'Wimbledon 1'),
    (2, 'Wimbledon 2'),
    (3, 'Wimbledon 3');
INSERT INTO players (player_id, first_name, last_name, team_id, group_id) VALUES
    (1, 'Aedan', 'Petty', 1, 1),
    (2, 'Aliza', 'Santos', 1, 3),
    (3, 'Kaylynn', 'Vaughan', 1, 6),
    (4, 'Arjun', 'Bauer', 1, 4),
    (5, 'Lilian', 'Huber', 1, 2),
    (6, 'Lizeth', 'Roberts', 1, 6),
    (7, 'Nathan', 'Mcdowell', 1, 7),
    (8, 'Alvin', 'Ali', 1, 8),
    (9, 'Jordin', 'Christensen', 1, 7),
    (10, 'Saul', 'Blevins', 1, 7),
    (11, 'Carina', 'Meza', 2, 2),
    (12, 'Isabelle', 'Campos', 2, 5),
    (13, 'Kyleigh', 'Phelps', 2, 1),
    (14, 'Angela', 'Wong', 2, 7),
    (15, 'Kole', 'Rojas', 2, 7),
    (16, 'Martha', 'Potts', 2, 4),
    (17, 'Tomas', 'Powell', 2, 6),
    (18, 'Paxton', 'Clarke', 2, 5),
    (19, 'Jamya', 'Dodson', 2, 8),
    (20, 'Georgia', 'Clements', 2, 8),
    (21, 'Edwin', 'Crawford', 3, 2),
    (22, 'Malachi', 'Osborn', 3, 8),
    (23, 'Zion', 'Kent', 3, 2),
    (24, 'Anahi', 'Reyes', 3, 5),
    (25, 'Maddox', 'Cabrera', 3, 8),
    (26, 'Brody', 'Gutierrez', 3, 4),
    (27, 'Hayley', 'Stevenson', 3, 3),
    (28, 'Kamora', 'Sanchez', 3, 8),
    (29, 'Livia', 'Holmes', 3, 6),
    (30, 'Tanner', 'Jenkins', 3, 8),
    (31, 'Madelyn', 'Meadows', 4, 5),
    (32, 'Paola', 'Wilkerson', 4, 3),
    (33, 'Jared', 'Patton', 4, 6),
    (34, 'Pierre', 'Washington', 4, 8),
    (35, 'Dominik', 'Cochran', 4, 4),
    (36, 'Miya', 'Skinner', 4, 4),
    (37, 'Mara', 'Barnett', 4, 5),
    (38, 'Cornelius', 'Dodson', 4, 2),
    (39, 'Ashleigh', 'Kaiser', 4, 6),
    (40, 'Weston', 'Meza', 4, 6);
-- scores deliberately NULL here; set after goals load so the sync trigger validates
INSERT INTO matches (match_id, match_datetime, home_team_id, away_team_id, venue_id, match_type,
home_score, away_score) VALUES
    (1, '2022-10-01 16:00:00', 1, 2, 1, 'league', NULL, NULL),
    (2, '2022-10-01 16:00:00', 3, 4, 2, 'league', NULL, NULL),
    (3, '2022-10-08 16:00:00', 1, 3, 2, 'league', NULL, NULL),
    (4, '2022-10-08 16:00:00', 2, 4, 3, 'league', NULL, NULL),
    (5, '2022-10-22 16:00:00', 1, 4, 3, 'league', NULL, NULL),
    (6, '2022-10-22 16:00:00', 2, 3, 2, 'league', NULL, NULL),
    (7, '2022-10-29 16:00:00', 2, 1, 1, 'league', NULL, NULL),
    (8, '2022-10-29 16:00:00', 4, 3, 3, 'league', NULL, NULL),
    (9, '2022-11-05 16:00:00', 3, 1, 3, 'league', NULL, NULL),
    (10, '2022-11-05 16:00:00', 4, 2, 1, 'league', NULL, NULL),
    (11, '2022-11-12 16:00:00', 4, 1, 2, 'league', NULL, NULL),
    (12, '2022-11-12 16:00:00', 3, 2, 3, 'league', NULL, NULL),
```

```
    (13, '2023-11-12 16:00:00', 4, 1, 1, 'friendly', NULL, NULL),
    (14, '2023-11-12 16:00:00', 3, 2, 3, 'friendly', NULL, NULL);
INSERT INTO goals (goal_id, match_id, player_id, minute) VALUES
    (1, 1, 8, 5),
    (2, 1, 7, 80),
    (3, 2, 27, 11),
    (4, 2, 35, 10),
    (5, 3, 8, 2),
    (6, 3, 8, 88),
    (7, 3, 27, 85),
    (8, 4, 19, 20),
    (9, 4, 17, 40),
    (10, 4, 35, 30),
    (11, 6, 19, 5),
    (12, 6, 28, 2),
    (13, 6, 27, 55),
    (14, 7, 19, 1),
    (15, 7, 16, 82),
    (16, 7, 8, 35),
    (17, 7, 3, 77),
    (18, 8, 35, 66),
    (19, 8, 25, 12),
    (20, 8, 27, 68),
    (21, 9, 29, 20),
    (22, 9, 29, 25),
    (23, 9, 26, 85),
    (24, 9, 7, 77),
    (25, 9, 8, 89),
    (26, 10, 19, 75),
    (27, 11, 35, 25),
    (28, 11, 37, 46),
    (29, 12, 26, 80),
    (30, 12, 25, 90),
    (31, 12, 19, 29),
    (32, 12, 18, 87);
INSERT INTO cards (card_id, match_id, player_id, card_type, minute) VALUES
    (1, 1, 12, 'yellow', 10),
    (2, 2, 30, 'yellow', 30),
    (3, 3, 1, 'yellow', 10),
    (4, 3, 9, 'yellow', 70),
    (5, 4, 32, 'red', 40),
    (6, 5, 10, 'yellow', 35),
    (7, 5, 38, 'yellow', 88),
    (8, 7, 3, 'yellow', 60),
    (9, 8, 35, 'yellow', 10),
    (10, 9, 23, 'yellow', 30),
    (11, 10, 32, 'red', 10),
    (12, 11, 1, 'yellow', 62),
    (13, 12, 17, 'yellow', 42);
-- apply real scores now that goal rows exist; sync trigger re-checks each
UPDATE matches AS m SET home_score = v.home_score, away_score = v.away_score
FROM (VALUES
    (1, 2, 0),
    (2, 1, 1),
    (3, 2, 1),
    (4, 2, 1),
    (5, 0, 0),
    (6, 1, 2),
    (7, 2, 2),
    (8, 1, 2),
    (9, 3, 2),
    (10, 0, 1),
    (11, 2, 0),
    (12, 2, 2)
) AS v(match_id, home_score, away_score)
WHERE m.match id = v.match id;

-- 'GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY' means we can SPECIFY the id's
-- advance each identity sequence past the ids we loaded by hand,
-- so any future INSERT gets a new unique id
-- since we manually specfied ID's for each primary key column and
-- the default behaviour is auto-generating the values,
-- here we tell postgreSQL to start its auto-generating counter from the current highest
-- to ensure new inserts start from a new ID number rather than a previously used one.
SELECT setval(pg_get_serial_sequence('teams','team_id'), (SELECT MAX(team_id)   FROM teams));
SELECT setval(pg get serial sequence('groups','group id'), (SELECT MAX(group id)  FROM groups));
SELECT setval(pg_get_serial_sequence('venues','venue_id'),   (SELECT MAX(venue_id)  FROM venues));
SELECT setval(pg get serial sequence('players','player id'), (SELECT MAX(player id) FROM players));
SELECT setval(pg_get_serial_sequence('matches','match_id'),  (SELECT MAX(match_id)  FROM matches));
SELECT setval(pg_get_serial_sequence('goals','goal_id'),     (SELECT MAX(goal_id)   FROM goals));
```

```
SELECT setval(pg_get_serial_sequence('cards','card_id'),     (SELECT MAX(card_id)   FROM cards));


-- DATA VERIFICATION
-- VERIFY THE DATA LOADED IN CORRECTLY (can run this to check all the data is present)
SELECT 'teams' AS table_name, COUNT(*) AS row_count FROM teams
UNION ALL SELECT 'groups',  COUNT(*) FROM groups
UNION ALL SELECT 'venues',  COUNT(*) FROM venues
UNION ALL SELECT 'players', COUNT(*) FROM players
UNION ALL SELECT 'matches', COUNT(*) FROM matches
UNION ALL SELECT 'goals',   COUNT(*) FROM goals
UNION ALL SELECT 'cards',   COUNT(*) FROM cards
ORDER BY table_name;

-- VERIFY THAT THE FEEDBACK FIXES ACTUALLY WORK
-- We intentionally try to insert data that is not possible within the specified constraints to
-- test said constraints.
-- Event-player rule: player 21 (Vis Wizards) never played in match 1 (Data Masters v BI Gods)
INSERT INTO goals (goal_id, match_id, player_id, minute) VALUES (999, 1, 21, 10);
-- score-sync rule: match 1 has two home goals, so forcing 5-0 must be rejected
UPDATE matches SET home_score = 5 WHERE match_id = 1;
```